

Scoring model

Comp Builder — how player, champion, role, synergy, and meta signals are weighed

How the comp builder turns raw records into the win-rate estimate on the score card. All of this lives in the `--- scoring model ---` section of `index.html` (constants around line 778, functions below them); the worker and fetch scripts only supply the data it reads.

The estimate for a full five-man comp is:

```
compScore = mean(slot scores of the 5 picks) +  $\Sigma$  pairwise synergy adjustments
```

clamped to [0.05, 0.95] (`compScore`, `index.html`). Every term below is in win-rate units — an adjustment of +0.02 means "worth about two percentage points of win rate".

A single pick's score is:

```
slotScore(player, champ, role) = indScore(player, champ, role) + dAdj(champ, role)
```

`indScore` is draft-independent and cached; `dAdj` is the draft-mode lane matchup nudge.

1. Individual score — `indScore(player, champ, role)`

The base is a **shrunk win rate**: the player's record on the champ, pulled toward a meta prior so small samples don't lie.

The record (role weighting)

Games the player played this champ **in this role** count at full weight. Games on the same champ **in other roles** count at `OFF_ROLE_GAME_WEIGHT = 0.3` (mechanics transfer between roles, but a jungle record shouldn't fully back a top-lane pick). With the **Strict role WRs** toggle on, the off-role weight halves to `0.15` instead of dropping to zero — a bad record elsewhere is still evidence, and zeroing it let demonstrably-losing champs score as fresh unknowns.

```
g = roleGames + (totalGames - roleGames) × offRoleWeight  
w = roleWins + (totalWins - roleWins) × offRoleWeight
```

Shrinkage toward the meta prior

```
wr = (w + prior × K_IND) / (g + K_IND)      K_IND = 8
```

Equivalent to adding 8 phantom games at the meta win rate. A player with 0 games scores exactly the prior; ~8 games is the halfway point where personal record and meta pull equally; by ~30 games the personal record dominates.

The meta prior — `metaWR(champ, role)`

From OP.GG current-patch stats (`fetch_meta.py` locally, daily cron in the worker):

1. If the champ has ≥ 100 **meta games in this role**, use that role's win rate.
2. Otherwise use the champ's overall win rate **minus** `OFF_META_PRIOR_PENALTY = 0.04` — the champ exists on this patch but isn't played here (Annie bot), so its overall WR is an optimistic prior.
3. No meta data at all $\rightarrow$ flat 0.5.

Flat adjustments on top

Term	Size	When
Mastery bonus	up to +0.03	linear in mastery points, capped at 200k (<code>masteryBonus</code>)
Off-role penalty	-0.04	the player has games on the champ but <25% of them are in this role (<code>OFF_ROLE_PENALTY</code>)
Role comfort	$\pm 0.12 \times$ (share - 0.2)	share = fraction of ALL the player's games in this role; 0.2 (one of five roles) is neutral. A 60%-jungle main gets $\sim +0.05$ in jungle and a small penalty elsewhere (<code>ROLE_COMFORT_WEIGHT</code>)

`indScore = wr + masteryBonus - offRolePenalty + roleComfort`

2. Draft matchup nudge — `dAdj(champ, role)` (draft mode only)

For each enemy pick that might lane against this champ:

```

edge = (0.5 - enemyWRvsUs) * play / (play + 50)    shrink by sample
edge = clamp(edge, ±MATCHUP_CAP)                  MATCHUP_CAP = 0.03
adj += laneProbability * edge

```

- Matchup records come from OP.GG counter tables via the worker's `/counters` endpoint (KV-cached a day); a record needs ≥ 20 **games** to count.
- `laneProbability` is `laneDist()`: the enemy champ's meta-viable lanes ($\geq 10\%$ role rate) weighted by how often it goes there, so a flex Yasuo splits its weight across mid/top/bot instead of betting on one lane. A manual lane tag on the enemy chip collapses this to certainty, and lanes claimed by other enemy picks are eliminated first.
- Each lane opponent contributes at most $\pm 3\text{pp}$, before the lane-probability weighting shrinks it further.

Separately, `hardCounters()` flags enemies with $\geq 50\%$ odds of being in the lane that beat the champ $\geq 55\%$ of the time (20+ games). **That flag never changes the score** — it raises a Swap/Keep prompt, and the champ only leaves the pick pool if the player opts in.

3. Synergy — pairwise adjustments in `compScore`

Two independent terms, summed over all 10 pairs in the comp. Both use heavier shrinkage than the individual term ($K_{\text{PAIR}} = 20$) because pair samples are tiny, both are scaled by $\text{SYNERGY_WEIGHT} = 0.5$, and both carry an extra confidence ramp $g / (g + 10)$ so a 2-game record moves the needle far less than a 20-game one.

Champ-pair synergy — `champPairAdj`

Pilot-attributed: only games where these two *players* piloted these two *champions* on the same team count (built in `mergeMatch / build_seed.py`; pairs need ≥ 2 games to be served at all). The record is measured against an expected WR — the mean of the two champs' individual meta win rates — so plain champion strength isn't double-counted as "synergy":

```
expected = (metaWR(a) + metaWR(b)) / 2
adj = (shrunk(wins, games, 20, expected) - expected) × 0.5 × games / (games + 10)
```

***Known weakness:** every champ-pair row in the current data sits at 2–5 games, so this term is mostly noise today. `META_SYNERGY_SPEC.md` specs replacing the flat expected baseline with OP.GG's champion-pair synergy stats (spec only, not implemented).*

Player-pair (duo) synergy — `playerPairAdj`

The two players' shared-game record vs a flat 50% expectation, role-weighted like `indScore`: shared games in exactly this role combo (e.g. this player jungle + that player mid) count fully, their other shared games at 0.3 (0 in strict mode). It also carries $\text{PAIR_OVERLAP_DISCOUNT} = 0.25$:

```
adj = (shrunk(w, g, 20) - 0.5) × 0.5 × 0.25 × g / (g + 10)
```

The 0.25 exists because the ten duo records in a 5-stack largely re-count the **same games** — summing them would multiply one shared win streak $\sim 10\times$; the discount makes ten fully-overlapping pairs count the group's record roughly once.

4. What gates a pick without weighing it

`candidates()` filters before scoring ever ranks:

- **Hard filters** — mastery range (default floor 30k points), games-played range, and draft availability (bans, enemy picks, fearless-session champs, champs locked by another ally, champs the player agreed to draft around after a hard-counter flag). Outside these a champ is simply not a pick.
- **Role viability** (`roleViable`) — the champ must have a real personal sample in the role (≥ 10 games) or real meta presence there (≥ 100 games and $\geq 10\%$ role rate). Soft: the candidate list degrades tier by tier rather than going empty — *viable* + ≥ 5 games played first, then *viable*, then anything eligible. In strict mode "played" counts in-role games only.
- A tiny tiebreaker ($+\text{roleShare} \times 0.02$) nudges candidate *ordering* toward champs the player actually plays in the role; it is not part of the displayed score.

5. What never feeds the score

- **Team profile panel** (damage mix / frontline / CC / shielding from `compute_profiles.py` and the worker's `deriveProfiles`) — advisory shape read-out only.
 - **Hard-counter flags** — prompt only, see §2.
 - **Fearless exclusions, bans, enemy picks** — availability, not weight.
-

Constants reference

Constant	Value	Meaning
K_IND	8	phantom games toward the meta prior, individual WR
K_PAIR	20	phantom games, both synergy terms
SYNERGY_WEIGHT	0.5	scale on both synergy deviations
PAIR_OVERLAP_DISCOUNT	0.25	de-duplicates overlapping duo records
MASTERY_MAX_BONUS	0.03	mastery bonus cap (at 200k points)
OFF_ROLE_PENALTY	0.04	champ rarely played in this role (<25%)
OFF_META_PRIOR_PENALTY	0.04	prior haircut when meta doesn't play the champ here
OFF_ROLE_GAME_WEIGHT	0.3	off-role game weight (0.15 in strict mode)
ROLE_COMFORT_WEIGHT	0.12	scale on player's role-share deviation from 0.2
MIN_CHAMP_GAMES	5	"played" tier threshold in candidates
OWN_ROLE_VIABLE_GAMES	10	personal games making an off-meta role pickable
MATCHUP_CAP	0.03	per-lane-opponent matchup cap
HARD_COUNTER_WR	0.55	flag threshold (prompt only)